

Semantic Search & Retrieval

A standalone, in-progress reference for Build 3 — Phases 1-2 in full, every file, the full verification log, and a dedicated look at how a question actually flows through the agent at runtime.

STATUS	COMMITTS	CONTINUES	DATA CHOICE
Phases 1-2 complete, in progress	2e36f06 → f4f2eb4	Build 2 / Build 2.5	clean.allocations.summary

- What & Why
- Phase 1
- Phase 2
- Request Flow
- Verification Log
- Known Limitations
- Environment State

What Was Built, and Why

Build 2 gave the agent structured data to read. Build 3 gives it **semantic** reach over the one genuinely unstructured field in the schema — **clean.allocations.summary**, free-text equipment descriptions — so questions about the general kind or theme of a checkout ("camera gear") work even without exact keyword matches.

Data choice was deliberated, not assumed: two alternatives (a new purpose-sourced document corpus, and a meta/self-referential RAG agent over this project's own documentation) were considered and explicitly deferred — the documentation angle is logged as a future **Build 3.5. summary** won because it extends the same dataset across builds rather than sprawling into a new kind of corpus, per this project's own stated principle.

No new database role was needed here, unlike Build 2.5 — semantic search is just another *read*, so it uses **appdb_reader** like every other query tool.

1 Enable Semantic Search

Goal: get pgvector running, embed the existing corpus, and prove raw similarity search works before touching the agent at all.

Infrastructure change: docker-compose.yml's postgres image swapped from postgres:17 to pgvector/pgvector:pg17 — same Postgres 17, extension pre-installed. Data survives the swap (it lives in the separate pgdata volume, not the image). The swap did trigger a collation-version warning (the new image bundles a different glibc/ICU version) — resolved with `ALTER DATABASE appdb REFRESH COLLATION VERSION;`, safe for this dataset's plain ASCII/business text.

```
008_add_summary_embedding.sql
```

```
-- adds a pgvector embedding column for clean.allocations.summary.
-- Depends on the vector extension (Phase 1, step 1) and clean.allocations existing.
```

```
ALTER TABLE clean.allocations
  ADD COLUMN IF NOT EXISTS summary_embedding vector(384);
```

Embedding model choice: sentence-transformers (all-MiniLM-L6-v2, 384-dim), run locally — no new paid API/account, zero marginal cost, sufficient quality for equipment-description text. Considered and declined: Voyage AI (Anthropic's own recommended embeddings partner) and OpenAI's text-embedding-3-small — both good options, both would have added a new credential for a task that doesn't need frontier-tier embeddings.

```
embed_summaries.py
```

```
import os
from dotenv import load_dotenv
import psycopg
from pgvector.psycopg import register_vector
from sentence_transformers import SentenceTransformer

load_dotenv(encoding="utf-8-sig")

model = SentenceTransformer("all-MiniLM-L6-v2")

conn = psycopg.connect(
    host=os.environ.get("POSTGRES_HOST", "127.0.0.1"),
    port=5432,
    dbname=os.environ["POSTGRES_DB"],
    user=os.environ["POSTGRES_USER"],
    password=os.environ["POSTGRES_PASSWORD"],
)
```

```

register_vector(conn)

with conn.cursor() as cur:
    cur.execute("SELECT allocation_id, summary FROM clean.allocations")
    rows = cur.fetchall()

texts = [summary if summary else "" for _, summary in rows]
embeddings = model.encode(texts, show_progress_bar=True)

with conn.cursor() as cur:
    cur.executemany(
        "UPDATE clean.allocations SET summary_embedding = %s WHERE allocation_id = %s",
        [(embedding, allocation_id) for (allocation_id, _), embedding in zip(rows, embeddings)],
    )
conn.commit()
conn.close()

```

Verified: all 1,535 rows embedded, correct 384 dimensions, self-distance = 0 for spot-checked rows (a vector's distance to itself must always be zero — confirms the vector math itself, not just non-null values).

```

search_summaries.py - standalone verification script

query_embedding = model.encode(query_text)
cur.execute("""
    SELECT allocation_id, summary, summary_embedding <=> %s AS distance
    FROM clean.allocations
    ORDER BY distance
    LIMIT 5
""", (query_embedding,))

```

Relevance checkpoint passed via contrast test: "camera equipment" scored 0.469-0.513, returning genuinely coherent video/audio production gear. A deliberately out-of-domain query, "office desk chair" (this corpus has zero office furniture), scored meaningfully worse at 0.657-0.773 — real discrimination, even though the office-chair query's own top-5 results were still nonsense (there's nothing relevant to find; a plain top-K query has no "no good match" concept). That gap became the seed for Phase 2's threshold work.

2 Wire Into the Agent

Goal: expose semantic search as a real tool, with an actual relevance threshold instead of always returning top-K regardless of quality.

Calibration methodology correction, mid-phase: a first calibration attempt sampled real corpus summaries as "in-domain queries" and got a misleadingly large separation (0.16 vs 0.65) — because many real summaries are near-duplicates of each other (the same standard kit checked out repeatedly), this measured duplication, not relevance. Using that number as the threshold would have rejected Phase 1's own working example ("camera equipment" at 0.469-0.513 is *higher* than the flawed 0.4066 midpoint). Recalibrated using short natural-language queries on **both** sides — matching how the agent actually queries — giving a real, narrower, correctly-placed separation.

calibrate_threshold.py - corrected methodology

```
in_domain_queries = [
    "camera equipment", "video tripod", "audio recording gear", "lighting kit",
    "memory card", "laptop charger", "camera lens", "microphone", "HDMI cable",
    # ... 20 total, short phrases for things this corpus genuinely has
]
out_of_domain_queries = [
    "office desk chair", "kitchen blender", "garden shovel", "running shoes",
    # ... 20 total, short phrases for things this corpus has none of
]
# For each query: embed it, find its single nearest neighbor's distance in the corpus.
# Result: in-domain 0.3109-0.6019 (mean 0.4519) vs out-of-domain 0.6538-0.8235 (mean 0.7584)
# Clean separation - midpoint threshold ~0.6279, rounded to 0.63
```

tools.py - search_summaries

```
_embedding_model = None

def _get_embedding_model():
    global _embedding_model
    if _embedding_model is None:
        _embedding_model = SentenceTransformer("all-MiniLM-L6-v2")
    return _embedding_model

SIMILARITY_DISTANCE_THRESHOLD = 0.63

def search_summaries(query_text: str, limit: int = 5) -> list[dict]:
    model = _get_embedding_model()
    query_embedding = model.encode(query_text)
    # ... connects as appdb_reader, register_vector(conn) ...
    cur.execute("""
        SELECT allocation_id, summary, summary_embedding <=> %s AS distance
```

```

        FROM clean.allocations
        WHERE summary_embedding <=> %s < %s
        ORDER BY distance
        LIMIT %s
        """ , (query_embedding, query_embedding, SIMILARITY_DISTANCE_THRESHOLD, limit))
    rows = cur.fetchall()

    if not rows:
        return [{"message": f"No sufficiently relevant results for {query_text!r} ..."}]
    return [{"allocation_id": aid, "summary": summary, "distance": round(float(dist), 4)} for aid,
    summary, dist in rows]

```

Real incident during agent wiring: the `elif block.name == "search_summaries":` dispatch branch was missed when adding the tool schema. Because Python's `if/elif` chain had no matching branch and no `else`, the loop variable `result` silently kept its value from the *previous* tool call (`describe_table`'s column list) — no exception, so nothing looked broken. The model received a mismatched result and pragmatically fell back to a plain SQL `ILIKE '%camera%'` query instead, producing a correct-looking final answer that never actually exercised `search_summaries` at all. Caught only by checking that the tool's actual output matched its own logic, not by the final answer looking plausible.

agent.py – the fix

```

elif block.name == "search_summaries":
    result = search_summaries(block.input["query_text"])
else:
    raise ValueError(f"Unknown tool name: {block.name}")

```

That `else` is the generalizable part of the fix — any future tool added without its dispatch branch now fails loudly, instead of silently reusing stale data from whatever ran before it.

Verified live, twice: first pass (before the fix) proved the bug — the printed "result" for `search_summaries` was actually `describe_table`'s stale output. Second pass (after the fix) showed the real thing: a model-loading progress bar (first genuine invocation lazy-loads the embedding model) followed by the correct `{allocation_id, summary, distance}` shape, matching Phase 1's own numbers exactly for the same query. The agent then combined those results with a follow-up `run_sql_query` for exact checkout dates — a two-tool combination nobody coded explicitly, just the model composing what the system prompt made available.

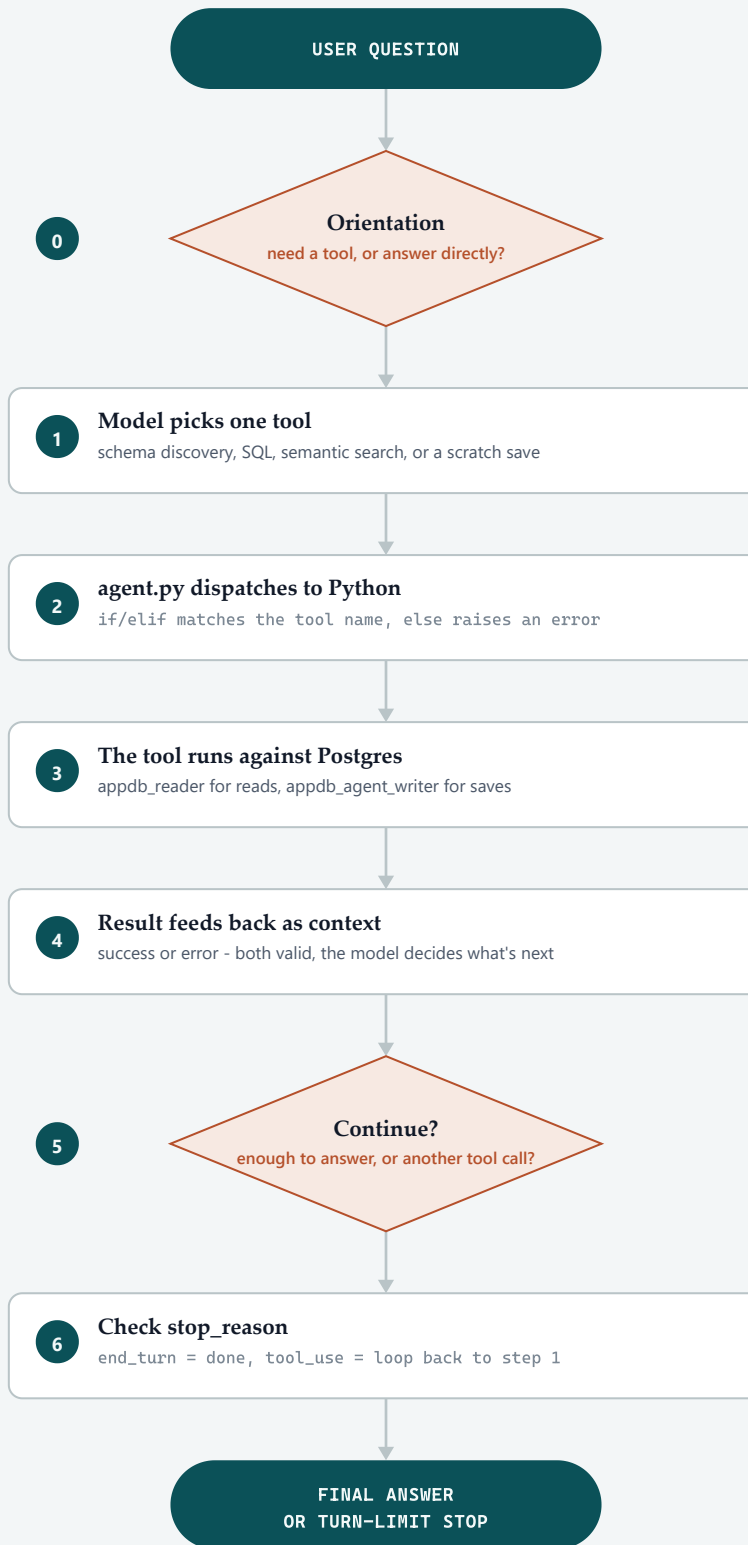
How a Question Flows Through the Agent

A dedicated walkthrough of the runtime request lifecycle as it stands after Phase 2 — distinct from the build-phase diagrams above, this is the architecture of a single question's journey, every time.

Build 3 — How a Question Flows Through the Agent

AGENT REQUEST LIFECYCLE · BUILD 3

● start / end ◇ checkpoint □ step → flow



Same visual system as the build/phase diagrams — this one documents runtime behavior, not a build plan.

1. The question enters as a single message. `user_question` is wrapped into `messages = [{"role": "user", "content": user_question}]` — that's the entire starting context. The model gets no hardcoded schema, no pre-fetched data — just the question, the `SYSTEM_PROMPT`, and the five tool schemas (`run_sql_query`, `list_tables`, `describe_table`, `save_dataframe`, `search_summaries`).

2. First API call — the model orients itself. Per the system prompt ("you don't know the schema in advance"), it typically calls `list_tables` first, then `describe_table` on whatever looks relevant — this is pure discovery, no data yet. Each of these returns as a `tool_use` block; the `while turn < MAX_TOOL_TURNS` loop dispatches it through the `if/elif/else` chain in `agent.py`, gets a result back from `tools.py`, and feeds that result back to the model as the next turn's context.

3. The real judgment call — which data tool to reach for. This is decided by the model itself, guided by the system-prompt line: *"For questions about the general kind or theme of equipment... use search_summaries instead of writing SQL yourself."* Nothing in the code forces this choice — it's the model reading that instruction and applying it. A structured question ("how many allocations did GART make?") gets `run_sql_query`. A thematic one ("what camera equipment...") gets `search_summaries`.

4. If it picks `search_summaries`: `tools.py` embeds the query text with the lazily-loaded `sentence-transformers` model (loaded once, cached module-level), runs a cosine-distance query against `clean.allocations.summary_embedding` filtered by the calibrated `0.63` threshold, and returns either real `{allocation_id, summary, distance}` rows or an explicit "no sufficiently relevant results" message — never a silent empty list.

5. Observed emergent pattern — semantic search feeds structured follow-up. In both live tests, the model didn't stop at `search_summaries`'s results — it took the returned `allocation_id`s and issued a *second*, precise `run_sql_query` (e.g., pulling exact `actual_start` / `actual_end` dates) to enrich the answer. Nobody coded that combination explicitly; it's the model composing two tools because the system prompt gave it both options and the question benefited from both.

6. Self-correction, observed again. When a query is written as `FROM allocations` (unqualified) instead of `FROM clean.allocations`, it fails with a real Postgres error — which comes back as a normal (non-crashing) `tool_result` with `is_error: True`, per Build 1's hardening pass. The model reads that error and retries schema-qualified, without any help.

7. Termination. The loop ends when `stop_reason` is `end_turn` (a plain-text final answer, no more tool calls) — or, as a safeguard, when `MAX_TOOL_TURNS` (10) is hit, printing a clean stop message instead of hanging forever.

Underneath all of this, the same trust boundary from Build 1 still governs everything, now with two lanes: the LLM proposes actions but never executes anything directly → `agent.py`'s dispatch is the gate (now with the `else: raise` guard against an unhandled tool silently reusing stale data) → the database role is the real backstop. Read paths (`run_sql_query`, `list_tables`, `describe_table`, `search_summaries`) all run as `appdb_reader` — physically incapable of writing. `save_dataframe` alone runs as `appdb_agent_writer` — physically incapable of touching anything outside `agent_scratch`. The

model's own judgment decides *which* tool to call; the roles decide what's *possible* regardless of what it decides.

Verification Log

✓ **Embedding backfill**

1,535/1,535 rows embedded, correct 384 dimensions, self-distance = 0 on spot checks.

✓ **Relevance contrast test**

"camera equipment" (0.469-0.513) vs "office desk chair" (0.657-0.773) — real, meaningful separation.

✓ **Threshold calibration**

Corrected methodology: in-domain 0.3109-0.6019 vs out-of-domain 0.6538-0.8235 → threshold 0.63, empirically justified not guessed.

✓ **search_summaries direct test**

Relevant query returns real rows; out-of-domain query returns the explicit "no relevant results" message, not an empty list.

✓ **Agent integration, twice**

First run exposed the missing-dispatch bug (stale result silently reused); second run, post-fix, showed the real tool executing (progress bar + matching distances) and combining with a follow-up SQL query for exact dates.

Known Limitations & Open Items

No automated re-embedding for new rows. `embed_summaries.py` is a one-time backfill. If the raw→clean pipeline ever runs again with new source data, new `clean.allocations` rows will have NULL `summary_embedding` until this script is re-run manually. Not automated - deliberately out of scope for this build.

The 0.63 threshold is an empirically-grounded heuristic, not a guarantee. Calibrated from 40 curated queries (20 in-domain, 20 out-of-domain) against this specific corpus - a reasonable, evidence-based starting point, but not something that generalizes automatically if the corpus or query style changes substantially.

Carried forward from Build 2.5: the `list_tables/describe_table` merge-into-one-call idea and the lack of a per-table row/size cap on `agent_scratch` both remain low-priority, unaddressed items.

Incidental fix during this build: `profile.py` was renamed to `profile_raw_data.py` after it was found silently shadowing Python's stdlib `profile` module, breaking an unrelated import deep in torch's own import chain. Unrelated to semantic search itself, logged separately in project memory.

Environment State

Component	Detail
Postgres image	pgvector/pgvector:pg17 (was postgres:17)
New extension	vector
New column	clean.allocations.summary_embedding vector(384)
New SQL file	008_add_summary_embedding.sql
Embedding model	sentence-transformers, all-MiniLM-L6-v2 (local, free)
New/changed Python	embed_summaries.py, search_summaries.py, calibrate_threshold.py, tools.py (+search_summaries), agent.py (+tool, +dispatch, +else guard, +prompt line)
Renamed	profile.py → profile_raw_data.py
Role used	appdb_reader (no new role needed for this build)

In progress — Phases 1-2 complete and verified. This brief will be updated if further Build 3 phases happen, the same way the Build 2.5 brief was updated after its table-cap fix. Ongoing cross-build command reference lives in `sql-test-01-cheatsheet.html`.